

DPRNN

- Paper: <https://arxiv.org/pdf/1910.06379>
- Code: <https://github.com/ShiZiqiang/dual-path-RNNs-DPRNNs-based-speech-separation/blob/master/models.py>

Được thiết kế để giải quyết các hạn chế của mô hình truyền thống khi xử lý **chuỗi dữ liệu rất dài**, đặc biệt trong bài toán tách giọng nói từ tín hiệu hỗn hợp. Ý tưởng cốt lõi của phương pháp này nằm ở việc **chia nhỏ chuỗi đầu vào thành các đoạn ngắn hơn (chunks)** và **kết hợp hai loại xử lý cục bộ và toàn cục** để tối ưu hóa hiệu quả tính toán và nắm bắt được cả thông tin ngắn hạn lẫn dài hạn.

Cách tiếp cận: DPRNN sử dụng hai loại RNN:

- **Intra-chunk RNN:** Xử lý cục bộ trong từng đoạn nhỏ, giúp nắm bắt thông tin ngắn hạn trong đoạn.
- **Inter-chunk RNN:** Xử lý mối liên hệ giữa các đoạn, giúp mô hình hóa thông tin dài hạn trên toàn chuỗi.

Segmentation

Mục tiêu của Segmentation

1. Giảm chiều dài chuỗi đầu vào cho RNN:

- Chuỗi tín hiệu dài L được chia thành các đoạn nhỏ kích thước K .
- Việc này làm giảm độ phức tạp của mô hình từ $O(L)$ xuống $O(\sqrt{L})$.

2. Bảo toàn thông tin tín hiệu:

- Sử dụng các đoạn chồng lấn với kích thước bước $P=K/2$, đảm bảo rằng mỗi mẫu tín hiệu đầu vào xuất hiện trong nhiều đoạn.

3. Tạo đầu vào 3D cho DPRNN:

- Sau khi phân đoạn, tín hiệu đầu vào được biểu diễn dưới dạng tensor 3 chiều với kích thước $[N, K, S]$, trong đó:

- N: Kích thước đặc trưng (feature size).
- K: Kích thước mỗi đoạn (chunk size).
- S: Số đoạn ($S = \lfloor 2L/K \rfloor + 1$).

Các bước thực hiện

1. Zero-padding chuỗi đầu vào:

- Các đoạn đầu tiên và cuối cùng được thêm giá trị 0 để đảm bảo toàn bộ tín hiệu được bao phủ.
- Điều này giúp các đoạn chồng lấn bao gồm đầy đủ thông tin tín hiệu.

```
def pad_segment(self, input, segment_size):
    batch_size, dim, seq_len = input.shape
    segment_stride = segment_size // 2
    rest = segment_size - (segment_stride + seq_len % segment_size)
    if rest > 0:
        pad = Variable(torch.zeros(batch_size, dim, rest)).type_as(input)
        input = torch.cat([input, pad], 2)
    pad_aux = Variable(torch.zeros(batch_size, dim, segment_stride)).type_as(input)
    input = torch.cat([pad_aux, input, pad_aux], 2)
    return input, rest
```

1. Chia tín hiệu thành các đoạn chồng lấn:

- Với mỗi đoạn có kích thước K và bước $P=K/2$, tín hiệu đầu vào $W \in \mathbb{R}^{N \times L}$ được chia thành $S = \lfloor 2L/K \rfloor$ đoạn.
- Kết quả là tập hợp các đoạn $\{D_1, D_2, \dots, D_S\}$, trong đó mỗi đoạn $D_s \in \mathbb{R}^{(N \times K)}$.

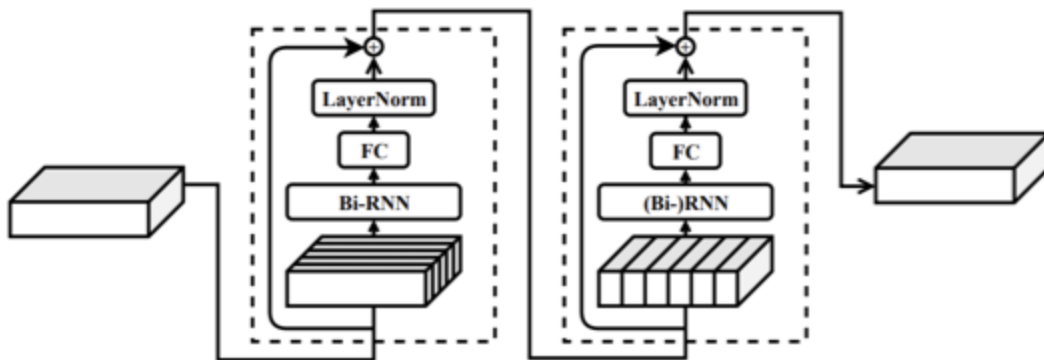
2. Sắp xếp thành tensor 3 chiều:

- Các đoạn được xếp lại thành tensor $T \in \mathbb{R}^{N \times K \times S}$, chuẩn bị cho các bước xử lý tiếp theo.
- $$T \in \mathbb{R}^{(N \times K \times S)}$$

```
def split_feature(self, input, segment_size):
    input, rest = self.pad_segment(input, segment_size)
    batch_size, dim, seq_len = input.shape
    segment_stride = segment_size // 2
    segments1 = input[:, :, :-segment_stride].contiguous().view(
    segments2 = input[:, :, segment_stride:].contiguous().view(
    segments = torch.cat([segments1, segments2], 3).view(batch_size,
    return segments.contiguous(), rest
```

Block Processing

Block Processing gồm hai thành phần chính: **Intra-chunk RNN** và **Inter-chunk RNN**.



Khối tensor ở bước segmentation sẽ có 3 chiều N, K, S . Intra RNN sẽ xử lý thông tin trên chiều K để nắm bắt thông tin cục bộ (trên 1 đoạn), còn inter RNN sẽ xử lý thông tin theo chiều S trên cùng 1 đơn vị thời gian K để nắm bắt thông tin toàn cục (liên kết giữa các đoạn)

Cấu trúc của Intra và Inter RNN

- BRNN: Có thể dùng RNN, LSTM
- FC: Chuyển đầu ra của RNN từ không gian đặc trưng H về không gian đầu vào ban đầu N (H là chiều ẩn xác định khả năng lưu trữ thông tin của mạng RNN)

- Layer Norm: chuẩn hóa LN để ổn định quá trình học
- **Residual Connection (Kết nối dư):**
 - Thêm đầu vào ban đầu vào đầu ra đã chuẩn hóa để giữ lại thông tin gốc

```
# dual-path RNN
class DPRNN(nn.Module):
    def __init__(self, rnn_type, input_size, hidden_size, output_size,
                  dropout=0, num_layers=1, bidirectional=True):
        super(DPRNN, self).__init__()

        # Khởi tạo các thành phần chính
        self.row_rnn = nn.ModuleList([]) # Intra-chunk RNN
        self.col_rnn = nn.ModuleList([]) # Inter-chunk RNN
        self.row_norm = nn.ModuleList([]) # Layer Normalization
        self.col_norm = nn.ModuleList([]) # Layer Normalization

        # Xây dựng các tầng RNN và chuẩn hóa
        for i in range(num_layers):
            # Intra-chunk RNN
            self.row_rnn.append(SingleRNN(rnn_type, input_size,
                                           hidden_size, dropout,
                                           self.row_norm.append(nn.GroupNorm(1, input_size, eps=1e-5))))

            # Inter-chunk RNN
            self.col_rnn.append(SingleRNN(rnn_type, input_size,
                                           hidden_size, dropout,
                                           self.col_norm.append(nn.GroupNorm(1, input_size, eps=1e-5))))

        # Lớp đầu ra cuối cùng
        self.output = nn.Sequential(nn.PReLU(),
                                     nn.Conv2d(input_size, output_size, kernel_size=1))

    def forward(self, input):
        """
        input shape: batch, N, dim1, dim2
        """
        batch_size, _, dim1, dim2 = input.shape
```

```

output = input

# Duyệt qua các tầng DPRNN
for i in range(len(self.row_rnn)):
    # Intra-chunk RNN: xử lý theo chiều dim1 (K)
    row_input = output.permute(0, 3, 2, 1).contiguous()
    row_output = self.row_rnn[i](row_input) # Áp dụng RNN
    row_output = row_output.view(batch_size, dim2, dim1)
    row_output = self.row_norm[i](row_output) # Chuẩn hóa
    output = output + row_output # Kết nối dư

    # Inter-chunk RNN: xử lý theo chiều dim2 (S)
    col_input = output.permute(0, 2, 3, 1).contiguous()
    col_output = self.col_rnn[i](col_input) # Áp dụng RNN
    col_output = col_output.view(batch_size, dim1, dim2)
    col_output = self.col_norm[i](col_output) # Chuẩn hóa
    output = output + col_output # Kết nối dư

# Đầu ra cuối cùng
output = self.output(output) # B, output_size, dim1, dim2
return output

```

Overlap-Add

Mục tiêu: Cộng các chunk đã được tách để xử lý ở bước block processing, trừ ra các kết nối dư (Residual Connection) đã thêm vào để tránh dư thừa dữ liệu

Các bước chính

1. Định vị trí đoạn:

- Mỗi đoạn D_s được định vị tại vị trí tương ứng trong tín hiệu gốc.
- Sử dụng tỉ lệ chồng lấn $P=K/2$, mỗi đoạn sẽ bắt đầu sau đoạn trước đó P bước.

2. Cộng chồng các đoạn:

- Các đoạn chồng lấn được "cộng lại" ở các vị trí giao nhau:

3. Cắt bỏ phần dư:

- Nếu tín hiệu đã được đệm thêm trong bước segmentation, phần đệm được loại bỏ để khôi phục tín hiệu ban đầu.

```
def overlap_and_add(signal, frame_step):
    """
    Overlap and add operation to reconstruct signal.
    Args:
        signal: Tensor, shape [B, N, L, K]
        frame_step: Integer, step size for overlap
    Returns:
        Reconstructed signal, shape [B, N, T]
    """
    # Get shape of the signal
    batch_size, n_channels, n_frames, frame_size = signal.shape
    signal = signal.permute(0, 1, 3, 2).contiguous() # [B, N, K, L]
    signal = signal.view(batch_size, n_channels * frame_size, n_frames)

    # Compute the full signal length
    full_length = frame_step * (n_frames - 1) + frame_size

    # Reconstruct signal using overlap-add
    result = torch.zeros(batch_size, n_channels, full_length).type_as(signal)
    for i in range(n_frames):
        start = i * frame_step
        end = start + frame_size
        result[:, :, start:end] += signal[:, :, i]

    return result
```